

Python3 – Cheat Sheet

Ονόματα Μεταβλητών

Έγκυρα	Παραδείγματα
Ξεκινούν με γράμματα ή κάτω παύλες (_), ακολουθούμενα από γράμματα, αριθμούς ή κάτω παύλες	<code>number</code> <code>_number</code> <code>_number_</code> <code>number1</code>
Μη έγκυρα	Παραδείγματα
Δεν μπορούν να ξεκινούν με αριθμό	<code>1number</code>
Δεν μπορούν να περιέχουν κενά	<code>first name</code>
Δεν μπορούν να είναι δεσμευμένες λέξεις-κλειδιά της Python	<code>if</code> <code>else</code> <code>elif</code> <code>for</code> <code>while</code> <code>and</code> <code>or</code> <code>not</code> <code>print</code> <code>from</code> <code>with</code> <code>break</code> <code>return</code>
Δεν μπορούν να περιέχουν κανένα από τους παρακάτω χαρακτήρες: <code>? % / : ' + -</code>	<code>first%number = 1</code>

Η Python είναι *case sensitive*: Η μεταβλητή **First_Name** είναι διαφορετική από τη μεταβλητή **first_name**.

Είσοδος Δεδομένων (input)

Μέσω της συνάρτησης **input()** η οποία επιτρέπει στους χρήστες να εισάγουν στοιχεία από το πληκτρολόγιο. Τα δεδομένα εισόδου μέσω της συνάρτησης **input()** αποθηκεύονται ως συμβολοσειρά (string).

Απλή είσοδος δεδομένων μέσω της **input()**

```
name = input("What's your name? ")
print("Hello {}".format(name))
```

Είσοδος δεδομένων μέσω της **input()** και έπειτα μετατροπή σε ακέραιο αριθμό

```
age = input("How old are you? ")
age = int(age)
```

```
if age >= 18:
    print("You can vote!")
else:
    print("You can't vote yet.")
```

Είσοδος δεδομένων μέσω της **input()** και απευθείας μετατροπή σε ακέραιο αριθμό

```
# Casting to numeric input can also be done directly, e.g.,
age = int(input("How old are you? "))
```

```
if age >= 18:
    print("You can vote!")
else:
    print("You can't vote yet.")
```

Τελεστές

Αριθμητικοί	Σχεσιακοί	Λογικοί
+ Πρόσθεση	> μεγαλύτερο από <i>Παραδείγματα:</i> $7 > 3 \Rightarrow \text{True}$ $7 > 15 \Rightarrow \text{False}$	not (αντιστρέφει την τιμή της λογικής έκφρασης) <i>Παραδείγματα:</i> $\text{not } (7 > 3) \Rightarrow \text{False}$ $\text{not } (7 > 15) \Rightarrow \text{True}$
- Αφαίρεση	< μικρότερο από <i>Παραδείγματα:</i> $7 < 3 \Rightarrow \text{False}$ $7 < 15 \Rightarrow \text{True}$	and (αν και οι δύο τελεστέοι είναι αληθείς, τότε η συνθήκη γίνεται αληθείς) <i>Παραδείγματα:</i> $(5 > 3) \text{ and } (7 > 3) \Rightarrow \text{True}$ $(7 \neq 15) \text{ and } (5 < 3) \Rightarrow \text{False}$
* Πολλαπλασιασμός	>= μεγαλύτερο ή ίσο με <i>Παραδείγματα:</i> $7 \geq 7 \Rightarrow \text{True}$ $7 \geq 15 \Rightarrow \text{False}$	or (αν κάποιος από τους τελεστέους είναι αληθείς, τότε η συνθήκη γίνεται αληθείς) <i>Παραδείγματα:</i> $(5 > 3) \text{ or } (7 > 3) \Rightarrow \text{True}$ $(7 \neq 15) \text{ or } (5 < 3) \Rightarrow \text{True}$ $(7 \neq 7) \text{ or } (5 < 3) \Rightarrow \text{False}$
/ Διάρθρωση κινητής υποδιαστολής <i>Παραδείγματα:</i> $5 / 2 = 2.5$ $5 / 3 = 1.6666666666666667$	<= μικρότερο ή ίσο με <i>Παραδείγματα:</i> $7 \leq 7 \Rightarrow \text{True}$ $7 \leq 3 \Rightarrow \text{False}$	
// Ακέραια διαίρεση (διαίρει το πρώτο όρισμα με το δεύτερο και στρογγυλοποιεί το αποτέλεσμα στον πλησιέστερο ακέραιο προς τα κάτω) <i>Παραδείγματα:</i> $5 // 2 = 2$ $5 // 3 = 1$	!= διάφορο του <i>Παραδείγματα:</i> $7 \neq 15 \Rightarrow \text{True}$ $7 \neq 7 \Rightarrow \text{False}$	
% Υπόλοιπο (Modulo) <i>Παραδείγματα:</i> $5 \% 2 = 1$ $5 \% 6 = 5$ $4 \% 2 = 0$	== ίσο με <i>Παραδείγματα:</i> $7 == 15 \Rightarrow \text{False}$ $7 == 7 \Rightarrow \text{True}$	
** Δύναμη <i>Παραδείγματα:</i> $5 ** 2 = 25$ $5 ** 3 = 125$		

Προτεραιότητα (Υψηλότερη προς Χαμηλότερη)

()
* *
* / % //
+ -
< <= >= > == !=
not
and
or
=

Σημειώσεις:

- Εάν υπάρχουν φωλιασμένες (nested) παρενθέσεις, η προτεραιότητα ξεκινά από την πιο εσωτερική παρένθεση προς την πιο εξωτερική, π.χ.

```
result = 5 * (x + y + (a * b))
```

- Εάν υπάρχουν πολλαπλές παρενθέσεις στο ίδιο επίπεδο, η προτεραιότητα ξεκινά από αριστερά προς τα δεξιά, π.χ.

```
result = 5 * (x + y) + 10 * (y - z)
```

- Για τους υπόλοιπους τελεστές (δηλαδή, **, *, /, %, //, +, -), εάν υπάρχουν πολλαπλοί τελεστές στο ίδιο επίπεδο, η προτεραιότητα ξεκινά από τα αριστερά προς τα δεξιά, π.χ.

```
result = 5 - 2 * 4 * y / x
```

Εκτύπωση Δεδομένων (print)

Μέσω της συνάρτησης **print()**, η οποία χρησιμοποιείται για την εκτύπωση του καθορισμένου μηνύματος στην οθόνη (ή σε άλλη τυπική συσκευή εξόδου).

Παραδείγματα:

- 1) Συμβολοσειρά (string) μόνο:

```
print("My favourite course is EPL031")
```

Έξοδος προγράμματος:

My favourite course is EPL031

- 2) Μεταβλητή και συμβολοσειρά (string):

```
my_course = "EPL031"  
print("My favourite course is", my_course)
```

Έξοδος προγράμματος:

My favourite course is EPL031

3) Μεταβλητή και συμβολοσειρά (string) μέσω συνένωσης συμβολοσειρών (string concatenation):

```
my_course = "EPL031"  
print("My favourite course is " + my_course)
```

Έξοδος προγράμματος:

My favourite course is EPL031

4) Μεταβλητή και συμβολοσειρά (string) μέσω της μορφοποίησης (string format):

```
my_course = "EPL031"  
print("My favourite course is {}".format(my_course))
```

Έξοδος προγράμματος:

My favourite course is EPL031

5) Μεταβλητή και συμβολοσειρά (string) μέσω f-string:

```
my_course = "EPL031"  
print(f"My favourite course is {my_course}")
```

Έξοδος προγράμματος:

My favourite course is EPL031

Στην προκαθορισμένη επιλογή, η συνάρτηση εκτύπωσης **print** τελειώνει με μια νέα γραμμή (new line). Εάν θέλουμε να αποτρέψουμε αυτή τη συμπεριφορά και να εκτυπώσουμε ένα μήνυμα στην ίδια γραμμή, πρέπει να ορίσουμε την παράμετρο **"end"** στη συνάρτηση print.

Παράδειγμα εκτύπωσης με την προκαθορισμένη επιλογή (τελειώνει με νέα γραμμή)

```
print("Hello")  
print("World")
```

Έξοδος προγράμματος:

Hello
World

Παράδειγμα εκτύπωσης χωρίς την προκαθορισμένη επιλογή (δεν τελειώνει με νέα γραμμή)

```
print("Hello", end=" ")  
print("World")
```

Έξοδος προγράμματος:

Hello World

Παράδειγμα εκτύπωσης χωρίς την προκαθορισμένη επιλογή (δεν τελειώνει με νέα γραμμή)

```
print("Hello", end="***")  
print("World")
```

Έξοδος προγράμματος:

Hello***World

Έλεγχος Ροής Προγράμματος (if-elif-else)

Δομή:

```
if συνθήκη1:
    # εντολές που θα εκτελεστούν εάν η συνθήκη1 είναι αληθής
elif συνθήκη2:
    # εντολές που θα εκτελεστούν εάν η συνθήκη2 είναι αληθής
# ...
elif συνθήκηN:
    # εντολές που θα εκτελεστούν εάν η συνθήκηN είναι αληθής
else:
    # εντολές που θα εκτελεστούν εάν καμία από τις
    # προηγούμενες συνθήκες δεν ήταν αληθής
```

Σημειώσεις:

- Το **if** είναι πάντα πρώτο. Το **if** μπορεί να υπάρξει αυτόνομο.
- Μπορούν να ακολουθήσουν όσα **elif** χρειάζονται, το καθένα με τη δική του συνθήκη (εάν χρειάζονται, δεν είναι υποχρεωτικά).
- Το **else** είναι πάντα τελευταίο (εάν χρειάζεται, δεν είναι υποχρεωτικό). Εάν θα χρησιμοποιηθεί, τότε πρέπει να είναι μόνο ένα.

Έλεγχος ροής μέσω του *if*

```
age = 19
```

```
if age >= 18:
    print("You're old enough to vote!")
```

Έξοδος προγράμματος:

```
You're old enough to vote!
```

Έλεγχος ροής μέσω του *if*

```
age = 17
```

```
if age >= 18:
    print("You're old enough to vote!")
```

Έξοδος προγράμματος:

ΠΡΟΣΟΧΗ! Σε αυτό το παράδειγμα, το πρόγραμμα δεν θα εκτυπώσει κάτι επειδή δεν ισχύει η συνθήκη του **if** καθώς επίσης δεν υπάρχει ούτε συνθήκη **else** για να εκτελεστούν άλλες εντολές.

Έλεγχος ροής μέσω του συνδυασμού *if-else*

```
age = 17
```

```
if age >= 18:
    print("You're old enough to vote!")
else:
    print("You can't vote yet.")
```

Έξοδος προγράμματος:

```
You can't vote yet.
```

Έλεγχος ροής μέσω του συνδυασμού if-elif-else

```
age = 10
price = 0

if age < 12:
    price = 5
elif age < 18:
    price = 10
else:
    price = 20

print("The price is {}".format(price))
```

Έξοδος προγράμματος:

The price is 5

Έλεγχος ροής μέσω του συνδυασμού if-elif-else

```
age = 15
price = 0

if age < 12:
    price = 5
elif age < 18:
    price = 10
else:
    price = 20

print("The price is {}".format(price))
```

Έξοδος προγράμματος:

The price is 10

Έλεγχος ροής μέσω του συνδυασμού if-elif-else

```
age = 25
price = 0

if age < 12:
    price = 5
elif age < 18:
    price = 10
else:
    price = 20

print("The price is {}".format(price))
```

Έξοδος προγράμματος:

The price is 20

Έλεγχος ροής – Ανεξάρτητα if

age = 19

```
if age >= 18:
    print("You're adult!")
else:
    print("You're underage.")

if age >= 18:
    print("You're old enough to vote!")
else:
    print("You can't vote yet.")
```

Έξοδος προγράμματος:

You're adult!

You're old enough to vote!

ΠΡΟΣΟΧΗ! Σε αυτό το παράδειγμα, τα δύο if είναι ανεξάρτητα. Πρώτα θα εκτελεστεί το πρώτο if και ακολούθως το δεύτερο if.

Έλεγχος ροής – Ανεξάρτητα if

age = 17

```
if age >= 18:
    print("You're adult!")
else:
    print("You're underage.")

if age >= 18:
    print("You're old enough to vote!")
else:
    print("You can't vote yet.")
```

Έξοδος προγράμματος:

You're underage.

You can't vote yet.

ΠΡΟΣΟΧΗ! Σε αυτό το παράδειγμα, τα δύο if είναι ανεξάρτητα. Πρώτα θα εκτελεστεί το πρώτο if και ακολούθως το δεύτερο if.

Βρόχοι Επανάληψης (Εκτέλεση εντολών μέσω επανάληψης)

Επανάληψη μέσω for

Χρησιμοποιείται όταν γνωρίζουμε εκ των προτέρων τον συνολικό αριθμό επαναλήψεων ή τις πραγματικές τιμές τις οποίες θα θέλαμε να επαναλάβουμε (π.χ. επανάληψη ενός συγκεκριμένου εύρους τιμών).

Δομή:

```
for variable in range_of_values:
    # εντολή1
    # εντολή2
    # ...
    # εντολήN
```

Εντολές με βαθύτερη στοίχιση

Κάθε εντολή με βαθύτερη στοίχιση θα εκτελεστεί **για κάθε τιμή** στο `range_of_values`. Σε κάθε επανάληψη, η τρέχουσα τιμή αποθηκεύεται στη μεταβλητή `variable`.

Δημιουργία Εύρους Τιμών μέσω της συνάρτησης `range`:

start: Ο αρχικός αριθμός του συνόλου τιμών που θα δημιουργηθεί. Είναι προαιρετική παράμετρος. Αν δεν χρησιμοποιηθεί, τότε ο αρχικός αριθμός θα είναι το 0.

stop: Οι αριθμοί που θα δημιουργηθούν θα είναι μέχρι εδώ αλλά **δεν θα περιλαμβάνουν αυτή την τιμή**.

step: Η διαφορά που θα έχουν οι αριθμοί μεταξύ τους. Είναι προαιρετική παράμετρος. Αν δεν χρησιμοποιηθεί, τότε η διαφορά που θα έχουν οι αριθμοί μεταξύ τους είναι 1.

(1) `range(start, stop)` *# η τιμή stop δεν περιλαμβάνεται*

Παράδειγμα:

`range(0, 10)` ➡ 0, 1, 2, ..., 9

(2) `range(stop)` *# Η προκαθορισμένη τιμή start είναι το 0*

Παράδειγμα:

`range(10)` ➡ 0, 1, 2, ..., 9

(3) `range(start, stop, step)` *# Η επόμενη τιμή αυξάνεται ή
μειώνεται σύμφωνα με την τιμή step*

Παράδειγμα:

`range(0, 10, 2)` ➡ 0, 2, 4, 6, 8

`range(10, 0, -2)` ➡ 10, 8, 6, 4, 2

Επανάληψη εντολών σε εύρος τιμών μέσω της συνάρτησης `range()`

```
for i in range(0, 10):
    print(i, end=" ")
```

Έξοδος προγράμματος:

0 1 2 3 4 5 6 7 8 9

Επανάληψη εντολών σε εύρος τιμών μέσω της συνάρτησης `range()`

```
for i in range(0, 10):  
    print(i)
```

Έξοδος προγράμματος:

0
1
2
3
4
5
6
7
8
9

Επανάληψη εντολών σε εύρος τιμών μέσω της συνάρτησης `range()` με αντίστροφη σειρά

```
for i in range(9, -1, -1):  
    print(i, end=" ")
```

Έξοδος προγράμματος:

9 8 7 6 5 4 3 2 1 0

Επανάληψη μέσω `while`

Χρησιμοποιείται όταν δεν γνωρίζουμε εκ των προτέρων τον συνολικό αριθμό των επαναλήψεων (π.χ. εξαρτάται από την είσοδο του χρήστη).

Δομή:

`while` συνθήκη:

```
# εντολή1  
# εντολή2  
# ...  
# εντολήN
```

Εντολές με βαθύτερη στοίχιση

Κάθε εντολή με βαθύτερη στοίχιση θα εκτελείται συνεχώς όσο η συνθήκη παραμένει αληθής.

Επανάληψη εντολών σε εύρος τιμών με χρήση `while`

```
i = 0
```

```
while i < 10:
```

```
    # ΠΡΟΣΟΧΗ! Πρέπει να ενημερώσουμε την τιμή της  
    # μεταβλητής μέσα στη συνθήκη (δηλ., την μεταβλητή i).  
    # Στο συγκεκριμένο παράδειγμα αυτό επιτυγχάνεται  
    # με το να αυξήσουμε την τιμή της μεταβλητής i, η οποία  
    # χρησιμοποιείται σαν μετρητής (δηλ. σε κάθε επανάληψη,  
    # η τιμή αυξάνεται κατά 1), ούτως ώστε κάποτε να  
    # σταματήσει να είναι αληθής η συνθήκη. Αλλιώς, θα  
    # καταλήξουμε σε ατέρμονα βρόχο (infinite loop).  
    print(i, end=" ")  
    i = i + 1
```

Έξοδος προγράμματος:

0 1 2 3 4 5 6 7 8 9

Επανάληψη εντολών μέχρι να δοθεί σωστή τιμή εισόδου

```
input_msg = "Please provide a number between 0 and 10: "
number = int(input(input_msg))

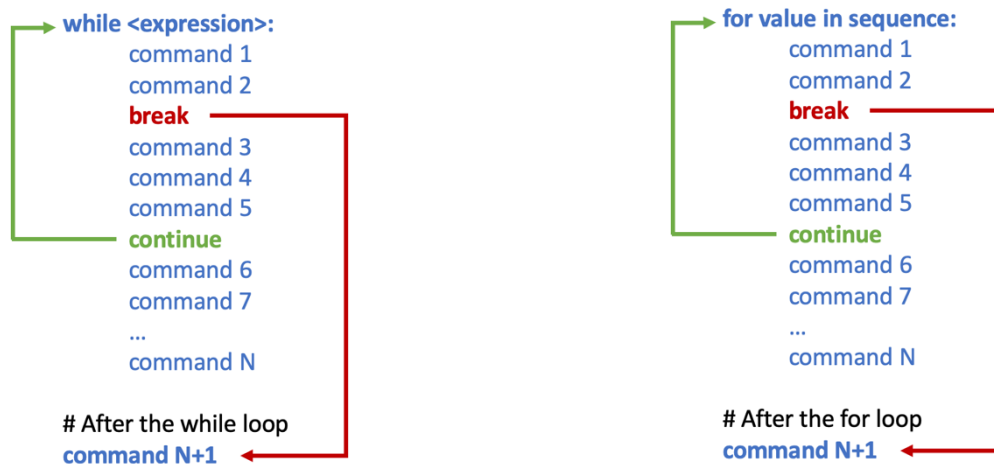
while number < 0 or number > 10:
    # ΠΡΟΣΟΧΗ! Πρέπει να ενημερώσουμε την τιμή της
    # μεταβλητής μέσα στη συνθήκη (δηλ., την μεταβλητή
    # number). Στο συγκεκριμένο παράδειγμα αυτό
    # επιτυγχάνεται με το να ζητήσουμε ξανά αριθμό από το
    # χρήστη μέσω της συνάρτησης input. Αλλιώς, θα
    # καταλήξουμε σε ατέρμονα βρόχο (infinite loop).
    number = int(input(input_msg))

print("Number provided: {}".format(number))
```

Εντολές break και continue

Η εντολή **break** μας επιτρέπει να βγούμε από την επανάληψη από την οποία την καλούμε, άσχετα με τη συνθήκη ελέγχου (while) ή τον αριθμό των επαναλήψεων που πρέπει να εκτελεστούν (for).

Η εντολή **continue** προκαλεί παράλειψη των υπολοίπων εντολών που υπάρχουν στο βρόχο επανάληψης (για την συγκεκριμένη επανάληψη) και εκ νέου εκτέλεση του σώματος εντολών.



Εντολή pass

Η εντολή **pass** χρησιμοποιείται όταν απαιτείται συντακτικά κάποια εντολή (π.χ. σε if-elif-else statements), αλλά δεν θέλετε να εκτελεστεί κάποια εντολή ή κώδικας στο συγκεκριμένο σημείο κώδικα. Η εντολή **pass** είναι μια μηδενική λειτουργία, δηλ. τίποτα δεν συμβαίνει όταν εκτελείται. Η εντολή **pass** είναι επίσης χρήσιμη σε μέρη όπου θα γράψετε κάποιο κώδικα, αλλά αυτό τον κώδικα δεν τον έχετε σκεφτεί ακόμα.

```
1
2 ilikia = int(input("Please give an age: "))
3
4 if ilikia > 70:
5     pass
6 elif ilikia > 18:
7     pass
8 elif ilikia < 1:
9     pass
10 else:
11     pass
12
13 print("Goodbye!")
14
```

Συμβολοσειρές (Strings)

Δείκτες/Θέσεις:

H	e	l	l	o		W	o	r	l	d
0	1	2	3	4	5	6	7	8	9	10
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
my_string = "Hello World"
print(my_string[0])      # H -> first character
print(my_string[-1])     # d -> last character
print(my_string[len(my_string)-1]) # d -> last character
print(my_string[11])     # IndexError: string index out of range
print(my_string[-12])    # IndexError: string index out of range
```

Επιλογή υπο-ακολουθίας συμβολοσειράς (Slicing)

[start_position:end_position]

start_position # Από ποια θέση να ξεκινήσει.
Η θέση συμπεριλαμβάνεται.

end_position # Σε ποια θέση να σταματήσει.
Η θέση ΔΕΝ συμπεριλαμβάνεται.

```
my_string = "Hello World"
print(my_string[0:5])      # Hello
print(my_string[6:11])     # World
print(my_string[4:9])      # o Wor
print(my_string[-11:-4])   # Hello W
```

Εάν δεν αναφέρουμε κάποιο από τους δείκτες, τότε η Python αυτόματα καταλαβαίνει ότι θέλουμε την πρώτη θέση ή την τελευταία θέση αντίστοιχα.

```
my_string = "Hello World"

# Εδώ παραλείπουμε την αρχική θέση, δηλ. θα επιστρέψει τους
# χαρακτήρες ξεκινώντας από την αρχή μέχρι την θέση 5 (Μη-
# συμπεριλαμβανομένης)
print(my_string[:5])      # Hello

# Εδώ παραλείπουμε την τελική θέση, δηλ. θα επιστρέψει τους
# χαρακτήρες ξεκινώντας από τη θέση 6 μέχρι το τέλος
print(my_string[6:])      # World
```

Τελεστές στις συμβολοσειρές

Τελεστής +

Ο τελεστής + σημαίνει συνένωση όταν χρησιμοποιείται με συμβολοσειρές.

```
my_string = "Hello" + " " + "World" + ". "
print(my_string)      # Hello World.
```

Τελεστής *

Ο τελεστής ***** σημαίνει επανάληψη όταν χρησιμοποιείται με συμβολοσειρές.

```
my_string = "Hello World. "  
print(my_string * 2)  # Hello World. Hello World.  
print(my_string * 3)  # Hello World. Hello World. Hello World.
```

Τελεστές σύγκρισης

Οι τελεστές **<**, **>**, **==**, **!=**, **<=**, **>=** συγκρίνουν συμβολοσειρές με βάση τον αριθμό της κωδικοποίησής τους στο σύνολο χαρακτήρων Unicode (Αλφαβητικά).

```
"a" < "b"      # True  
"A" < "B"      # True  
"a" < "A"      # False  
"ab" < "ba"    # True  
"aaab" < "aaac" # True. Διαφέρουν στο τέταρτο σύμβολο και  
ισχύει "b" < "c" για αυτό και "aaab" < "aaac"
```

Τελεστής in

Ο τελεστής **in** είναι τελεστής ελέγχου μέλους.

```
my_string = "Hello World."  
"He" in my_string      # True  
"k" in my_string       # False  
"ell" in my_string     # True  
"hello" in my_string   # False  
"orld" in my_string    # True
```

Μετατροπή αριθμού σε συμβολοσειρά

```
name = "Agent " + 123      # TypeError
```

Δεν επιτρέπεται να προσθέσω συμβολοσειρά με ακέραιο αριθμό. Θα πρέπει να μετατρέψω τον αριθμό σε συμβολοσειρά για να μπορώ να κάνω πρόσθεση 2 συμβολοσειρών. Αυτό επιτυγχάνεται μέσω της συνάρτησης **str()** η οποία θα μας δώσει τον αριθμό ως συμβολοσειρά.

```
id = 123  
name = "Agent " + str(id)  
print(name)      # Agent 123
```

Μετατροπή συμβολοσειράς σε αριθμό

Η συνάρτηση **int()** μετατρέπει την μεταβλητή τύπου string σε μεταβλητή ακέραιου τύπου (int).

```
str_value = "1234"  
int_value = int(str_value)
```

Η συνάρτηση **float()** μετατρέπει την μεταβλητή τύπου string σε μεταβλητή κινητής υποδιαστολής (float).

```
str_value = "12.34"  
float_value = float(str_value)
```

Εκτύπωση συμβολοσειράς μέσω επανάληψης (Διαπερνά την συμβολοσειρά και εκτυπώνει ένα-ένα τους χαρακτήρες)

```
my_string = "Hello"
```

```
for char in my_string:  
    print("Current character is: {}".format(char))
```

Έξοδος προγράμματος:

```
Current character is: H  
Current character is: e  
Current character is: l  
Current character is: l  
Current character is: o
```

Εκτύπωση συμβολοσειράς μέσω επανάληψης χρησιμοποιώντας τους δείκτες/θέσεις της συμβολοσειράς (Διαπερνά την συμβολοσειρά και εκτυπώνει ένα-ένα τους χαρακτήρες)

```
my_string = "Hello"
```

```
for i in range(0, len(my_string)):  
    print("Current character is: {}".format(my_string[i]))
```

Έξοδος προγράμματος:

```
Current character is: H  
Current character is: e  
Current character is: l  
Current character is: l  
Current character is: o
```

Εκτύπωση συμβολοσειράς μέσω επανάληψης (Διαπερνά την συμβολοσειρά και εκτυπώνει ένα-ένα τους χαρακτήρες)

```
my_string = "Hello"
```

```
i = 0
```

```
total_characters = len(my_string)
```

```
while i < total_characters:  
    print("Current character is: {}".format(my_string[i]))  
    i = i + 1 # ΠΡΟΣΟΧΗ! Εάν δεν ενημερώσουμε ορθά την τιμή  
             # της μεταβλητής i, τότε θα έχουμε ατέρμονα  
             # βρόχο επανάληψης (infinite loop).
```

Έξοδος προγράμματος:

```
Current character is: H  
Current character is: e  
Current character is: l  
Current character is: l  
Current character is: o
```

Μετατροπή πεζών γραμμάτων σε κεφαλαία και κεφαλαίων σε πεζά μέσω επανάληψης (Διαπερνώ την συμβολοσειρά, αντιστρέφω κάθε χαρακτήρα αναλόγως και τον "προσθέτω" σε μια καινούργια συμβολοσειρά)

```
user_str = input("Please give a string: ")
new_user_str = ""

for char in user_str:
    if char.isupper():
        char = char.lower()
    else:
        char = char.upper()

    new_user_str = new_user_str + char

print("The new string is {}".format(new_user_str))
```

Μετρητής φωνηέντων και συμφώνων (Διαπερνώ την συμβολοσειρά, ελέγχω κάθε χαρακτήρα εάν είναι φωνήεν ή όχι και προσθέτω + 1 στον ανάλογο μετρητή)

```
user_str = input("Please give a string: ")
counter_vowels = 0
counter_consonants = 0
vowels = "aeiou"

for char in user_str:
    char = char.lower()

    if char.isalpha():
        if char in vowels:
            counter_vowels = counter_vowels + 1
        else:
            counter_consonants = counter_consonants + 1

print("The string contains {} vowels and {} consonants".format(counter_vowels, counter_consonants))
```